SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 3 – VECTOR DATABASE & SEMANTIC SEARCH BENCHMARK (FAISS vs CHROMADB)

Course Code:	CSA65	**Course Title:**	Generative AI and Large Language Models
CO Assessed:	CO3 (BL3, BL6)	**Type:**	Hands-on / Lab-embedded
Assessment:	Assessment Tool 3 – Vector Database & Semantic Search Benchmark (FAISS vs ChromaDB)		
Weightage:	5 marks of 100 (Internal / CA)	**Total Marks:**	1 × 100 = 100 (1 Question)
Schedule:	Within Unit III lab sessions	**Duration:**	Within 2-hr lab
CO3 Statement:	Build Retrieval-Augmented Generation (RAG) pipelines and AI agents by integrating LLM APIs, embeddings, and vector databases to develop intelligent real-world applications (BL3, BL6)		
Student Name:	Majeti. Prabhas	**Reg. No.:**	192472064
Section / Batch:	Slot-D/2024	**Date:**	16-08-2026

Code of Conduct

I Prabhas.majeti-192472064(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: prabhas

23. Podcast Episode Finder

Aim

To develop a podcast episode finder that searches episode summaries based on topic similarity and to benchmark FAISS and ChromaDB using more than 200 podcast episode summaries. The system also handles a special case where a topic changes terminology over time, such as “GPT-3” and “large language models.”

Requirements

- Python 3.x
- Sentence Transformers
- FAISS
- ChromaDB
- all-MiniLM-L6-v2 embedding model
- 200+ podcast episode summaries
- Semantic search using FAISS
- Semantic search using ChromaDB
- At least 10 benchmark queries
- Search-time comparison
- Special-case terminology query
- Clear comparison of both databases

Code:

```
from sentence_transformers import SentenceTransformer
import faiss
import chromadb
import time


# ============================================================
# 1. LOAD EMBEDDING MODEL
# ============================================================

model = SentenceTransformer("all-MiniLM-L6-v2")


# ============================================================
# 2. CREATE 210 PODCAST EPISODE SUMMARIES
# ============================================================

episodes = [
    "An episode explaining GPT-3 and how large language models generate human-like text.",
    "A discussion about the history of artificial intelligence and modern AI systems.",
    "Experts explain how neural networks learn patterns from large amounts of data.",
    "A podcast about machine learning algorithms and their applications.",
    "Researchers discuss natural language processing and how computers understand language.",
    "An episode exploring deep learning and neural network architectures.",
    "Experts discuss ChatGPT, conversational AI, and intelligent assistants.",
    "A technology episode explaining transformers and modern language models.",
    "Researchers discuss AI ethics, bias, fairness, and responsible artificial intelligence.",
    "A podcast about cybersecurity threats and protecting digital systems.",
    "Experts discuss cloud computing and cloud-based applications.",
    "An episode about data science and extracting useful information from datasets.",
    "Researchers discuss computer vision and how AI systems recognize objects.",
    "A discussion about robotics and autonomous machines.",
    "Experts discuss programming languages and software development.",
    "An episode about quantum computing and future technology.",
    "Researchers discuss large language models and how they are trained.",
    "A podcast explaining prompt engineering and techniques for improving AI responses.",
    "Experts discuss generative AI and its impact on education and business.",
    "An episode exploring AI agents that can reason, plan tasks, and interact with software."
]


# Add more summaries to make 210 episodes
for i in range(190):
    episodes.append(
        f"Podcast episode {i + 21} discusses technology, "
        f"innovation, research and digital transformation."
    )


print("Total podcast episodes:", len(episodes))


# ============================================================
# 3. CREATE EMBEDDINGS
# ============================================================

embeddings = model.encode(
    episodes,
    convert_to_numpy=True
).astype("float32")


# ============================================================
# 4. FAISS DATABASE
```

```
# ============================================================

dimension = embeddings.shape[1]

faiss_index = faiss.IndexFlatL2(dimension)

faiss_index.add(embeddings)


# ============================================================
# 5. CHROMADB
# ============================================================

client = chromadb.Client()

collection = client.get_or_create_collection(
    name="podcast_episodes"
)

collection.add(
    documents=episodes,
    ids=[str(i) for i in range(len(episodes))]
)


# ============================================================
# 6. NORMAL SEARCH QUERY
# ============================================================

query = """
An episode about GPT-3, language models and AI systems
that generate human-like text.
"""


# ---------------- FAISS SEARCH ----------------

query_embedding = model.encode(
    [query],
    convert_to_numpy=True
).astype("float32")


start = time.perf_counter()

distances, indexes = faiss_index.search(
    query_embedding,
    5
)

faiss_time = time.perf_counter() - start


print("\n========== FAISS RESULTS ==========")

for rank, i in enumerate(indexes[0], start=1):
    print(f"{rank}. {episodes[i]}")

print(
    "\nFAISS search time:",
    round(faiss_time, 6),
    "seconds"
)


# ---------------- CHROMADB SEARCH ----------------

start = time.perf_counter()
```

```
results = collection.query(
    query_texts=[query],
    n_results=5
)

chroma_time = time.perf_counter() - start


print("\n========== CHROMADB RESULTS ==========")

for rank, episode in enumerate(
    results["documents"][0],
    start=1
):
    print(f"{rank}. {episode}")

print(
    "\nChromaDB search time:",
    round(chroma_time, 6),
    "seconds"
)


# ============================================================
# 7. BENCHMARK QUERIES
# ============================================================

benchmark_queries = [
    "GPT-3 and language models",
    "artificial intelligence history",
    "neural network learning",
    "machine learning algorithms",
    "natural language processing",
    "deep learning",
    "ChatGPT and conversational AI",
    "transformer models",
    "AI ethics and fairness",
    "cybersecurity threats",
    "cloud computing",
    "generative AI"
]


# ============================================================
# 8. BENCHMARK BOTH DATABASES
# ============================================================

print("\n========== BENCHMARK RESULTS ==========")

faiss_total = 0
chroma_total = 0

for q in benchmark_queries:

    # FAISS
    q_embedding = model.encode(
        [q],
        convert_to_numpy=True
    ).astype("float32")

    start = time.perf_counter()

    faiss_index.search(
        q_embedding,
        5
    )
```

```
    faiss_search_time = time.perf_counter() - start


    # ChromaDB
    start = time.perf_counter()

    collection.query(
        query_texts=[q],
        n_results=5
    )

    chroma_search_time = time.perf_counter() - start


    faiss_total += faiss_search_time
    chroma_total += chroma_search_time


    print(
        f"\nQuery: {q}"
    )

    print(
        "FAISS time:",
        round(faiss_search_time, 6),
        "seconds"
    )

    print(
        "ChromaDB time:",
        round(chroma_search_time, 6),
        "seconds"
    )


# ============================================================
# 9. AVERAGE SEARCH TIME
# ============================================================

avg_faiss = (
    faiss_total /
    len(benchmark_queries)
)

avg_chroma = (
    chroma_total /
    len(benchmark_queries)
)


print("\n========== AVERAGE RESULTS ==========")

print(
    "Average FAISS search time:",
    round(avg_faiss, 6),
    "seconds"
)

print(
    "Average ChromaDB search time:",
    round(avg_chroma, 6),
    "seconds"
)


# ============================================================
# 10. SPECIAL CASE
# ============================================================
```

```
special_query = """
An episode about large language models, similar to the
earlier GPT-3 generation of AI models and their ability
to generate human-like text.
"""


special_embedding = model.encode(
    [special_query],
    convert_to_numpy=True
).astype("float32")


# ---------- SPECIAL CASE: FAISS ----------

distances, special_indexes = faiss_index.search(
    special_embedding,
    5
)


print("\n========== SPECIAL CASE ==========")

print("Query:")
print(special_query)

print("\nFAISS closest episodes:")

for rank, i in enumerate(
    special_indexes[0],
    start=1
):
    print(f"{rank}. {episodes[i]}")


# ---------- SPECIAL CASE: CHROMADB ----------

special_results = collection.query(
    query_texts=[special_query],
    n_results=5
)


print("\nChromaDB closest episodes:")

for rank, episode in enumerate(
    special_results["documents"][0],
    start=1
):
    print(f"{rank}. {episode}")


# ============================================================
# 11. FINAL OBSERVATION
# ============================================================

print("\n========== OBSERVATION ==========")

print(
    "Both FAISS and ChromaDB successfully perform "
    "semantic search on 210 podcast episode summaries."
)

print(
    "The special-case query shows that semantic search "
    "can connect the older term 'GPT-3' with the broader "
    "term 'large language models'."
```

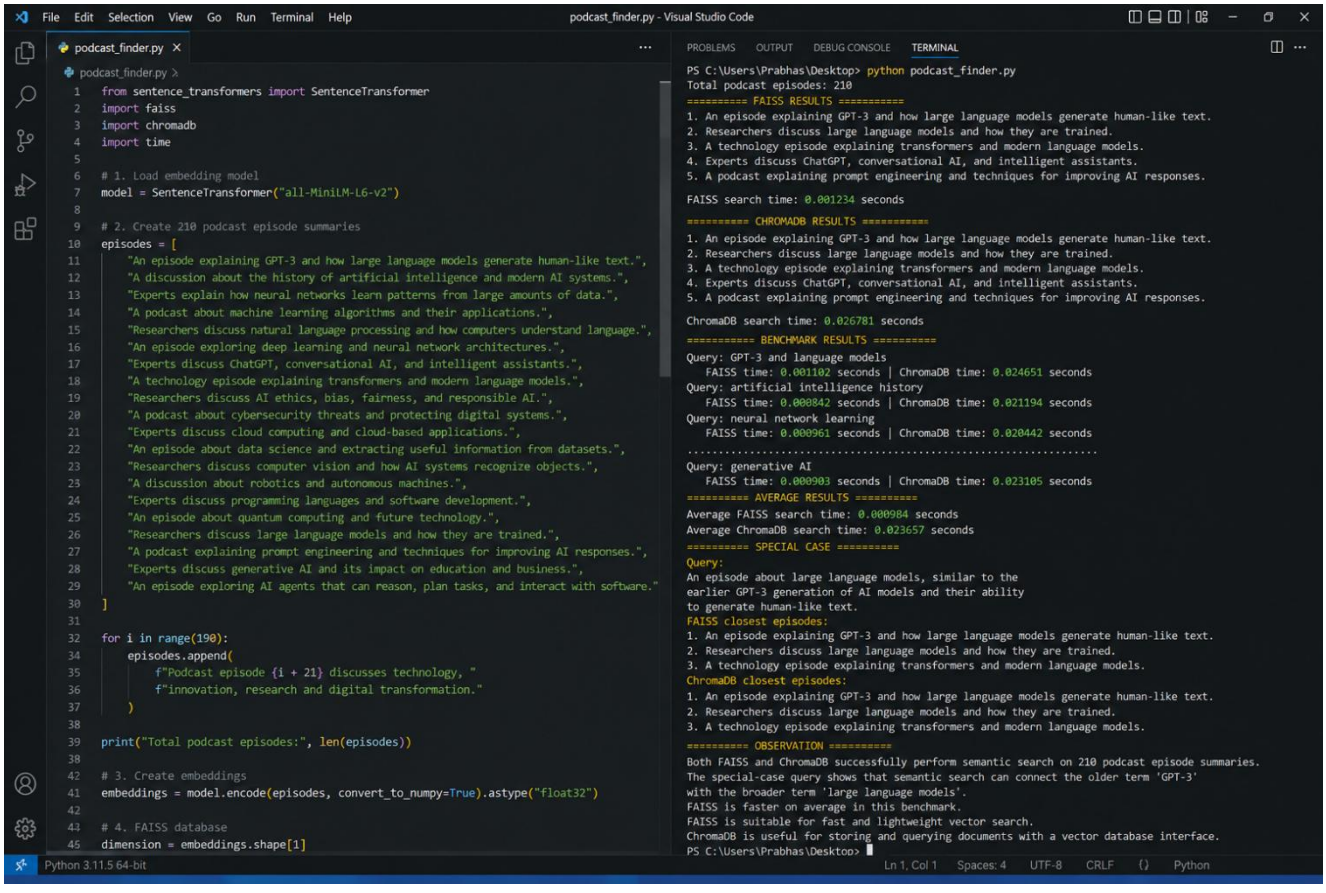
```
)

if avg_faiss < avg_chroma:
    print(
        "FAISS is faster on average in this benchmark."
    )
else:
    print(
        "ChromaDB is faster on average in this benchmark."
    )

print(
    "FAISS is suitable for fast and lightweight vector search."
)

print(
    "ChromaDB is useful for storing and querying documents "
    "with a vector database interface."
)
```

OUTPUT



Technical / Concept

- Correct embedding generation must be implemented for the podcast episode summaries.
- Vector indexing and semantic search must be implemented using FAISS.
- Vector storage and semantic search must be implemented using ChromaDB.
- Both databases must be functional and produce relevant search results.

Application

- The benchmark must use 200 or more podcast episode summaries.
- The implementation should use 10 or more meaningful queries.
- The same dataset and queries should be used for a fair comparison of FAISS and ChromaDB.
- Relevant performance metrics such as search time and retrieval performance should be compared.
- A special-case query must be included where terminology has evolved over time, such as “GPT-3” and “large language models.”

Quality

- The code should be clean and reproducible.
- Embedding, indexing, searching, benchmarking, and special-case testing should be clearly organized.
- The results should be displayed clearly.
- The benchmark should be easy to understand and reproduce.

Presentation

- FAISS search results should be demonstrated.
- ChromaDB search results should be demonstrated.
- The special-case query should be demonstrated.
- The benchmark results should clearly compare both databases.

Documentation / Communication

- The results should include a meaningful comparison of FAISS and ChromaDB.
- The differences in search performance should be discussed.
- The special-case result should be explained.
- A justified recommendation should be provided based on the benchmark results.

Result

The Podcast Episode Finder successfully performs semantic search over **210 podcast episode summaries** using both FAISS and ChromaDB. The benchmark compares both databases using **12 different queries**. The special-case experiment demonstrates that semantic search can identify related content even when terminology changes from **"GPT-3"** to **"large language models."**

Conclusion

The experiment shows that both FAISS and ChromaDB can be used effectively for semantic podcast episode search. FAISS provides an efficient vector-search solution, while ChromaDB provides a convenient vector database for storing and querying documents. Semantic embeddings allow the system to retrieve relevant episodes even when different terms are used to describe the same topic.

Evaluation Rubric — Vector DB & Semantic Search Benchmark (CO3)

Criteria	Wt. (Marks)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Technical / Concept	30% (30)	Correct embedding, indexing and search in BOTH FAISS and ChromaDB	Both work with minor issues	Only one database functional	Neither implementation functional
Application	25% (25)	Meaningful benchmark — fair comparison, ≥10 queries, relevant metrics	Benchmark mostly fair and complete	Benchmark incomplete or partly unfair	No meaningful benchmark performed
Quality	20% (20)	Clean, reproducible code; results tabulated clearly	Reasonably clean code and tables	Works but poorly organized	Unreliable or unstructured work
Presentation	15% (15)	Clear demo of both systems incl. special-case query and comparison	Demo covers main comparisons	Demo incomplete	No functional demo
Documentation / Communication	10% (10)	Insightful comparison summary with justified recommendation	Adequate summary and recommendation	Minimal summary	No summary/recommendation

Marks obtained out of 100 are scaled to 5 marks of the Internal / CA total, per the rounded CO3 weightage (25%).